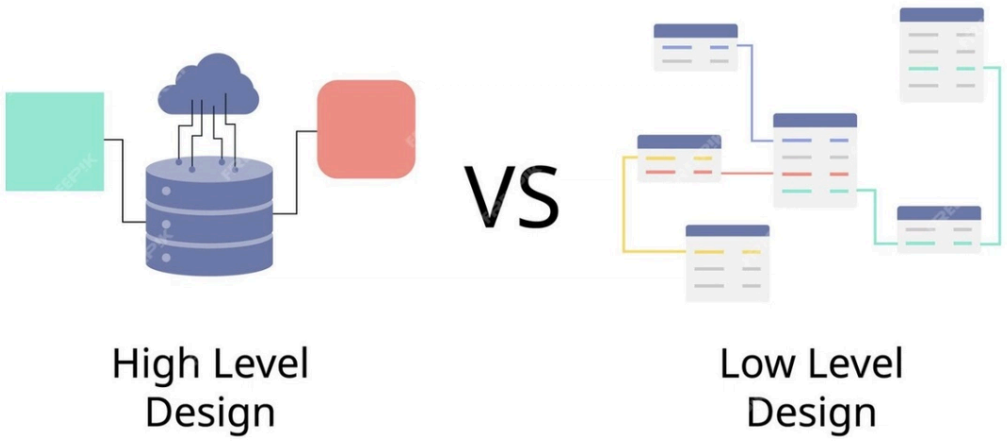
System design is the process of defining the architecture, components, modules, and interfaces of a system to meet specific requirements, as well as identifying the technologies and tools that will be used to implement the system.

Steps in the System Design process



Identify requirements

- Requirements gathering: Contacting customers, users, and stakeholders to understand their specific wants and needs.
- Requirements analysis: Review and categorize requirements into functional requirements (what the system must do) and non-functional requirements (performance, security, usability).

High-Level Design

High-Level Design (HLD) is a crucial step in the system design process, where we define the overall architecture and core structure of the system. The detailed steps include:

- Defining the system architecture: In this step, the overall architecture of the system is defined. The system is divided into major components, each with its own function. Use diagrams such as architectural diagrams, data flow diagrams (DFDs), and entity-relationship diagrams (ERDs) to represent the structure and relationships between the components.

- Identifying key components: After defining the architecture, the key components of the system are listed and described. Each key component is identified with a specific role and function within the system. The relationships and interactions between these components are also clarified.
- Requirements analysis and modeling: The functional and non-functional requirements of the system are analyzed in detail. These requirements are modeled using representational models such as Use Cases to describe the interaction between the user and the system.

Low-Level Design

Low-Level Design is the next step after HLD, where the components and modules of the system are detailed. These detailing steps include:

- Detailed component definition: Large components from the HLD are broken down into smaller modules. Each module is described in detail regarding how it works and interacts with the others. This ensures that all parts of the system are clearly defined and can be developed independently.
- Create class diagrams and sequence diagrams: Class diagrams are used to describe the internal structure of each module. Sequence diagrams describe how objects in the system interact through each step. These diagrams help ensure that all interactions in the system are clearly defined and executable.
- Defining interfaces and methods: The interfaces between modules and the methods used in the system are clearly defined. This helps ensure that all modules can interact with each other efficiently and without conflicts.
- Detailed design review and testing: Conduct simulation tests to ensure the design works as expected. Re-evaluate the design to find and fix any errors or weaknesses. This ensures that the final design is accurate and efficient.

Check and validate the design.

Conduct simulation tests to ensure the design works as expected. Re-evaluate the entire design to identify and fix errors or weaknesses.

Deployment and maintenance

Put the system into practical use. This process may include installing software, configuring the system, and training users. Continuously monitor, troubleshoot, and update the system to ensure it operates stably and meets new requirements.

Scalability

Scalability refers to a system's ability to handle increased load without compromising performance. To achieve this, engineers need to consider factors such as data partitioning, load balancing, and caching during the design process.

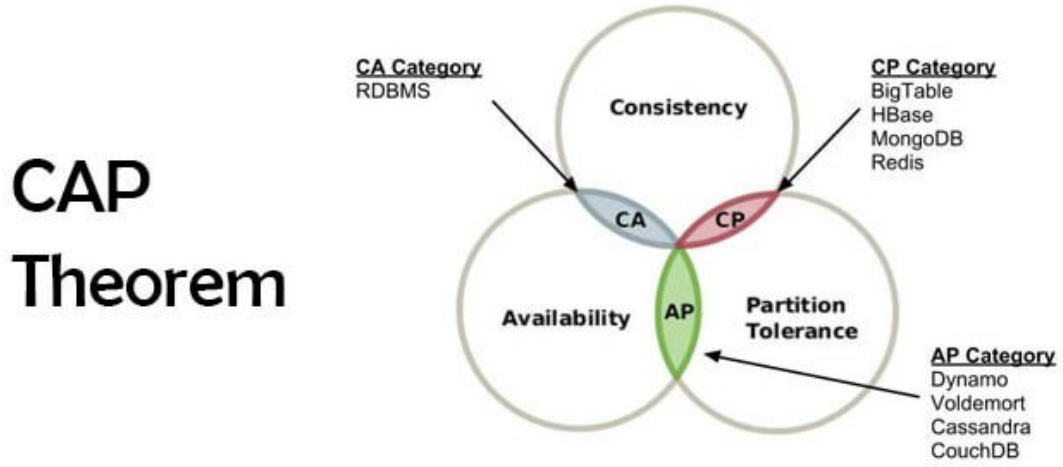
Availability

Availability, or availability, is the ability of a system to continue operating even when some components fail. Designing a system that ensures availability requires redundancy, failover, and fault tolerance.

Consistency

Consistency is a property of a system where all nodes see the same data at the same time. To ensure this property, you need to consider factors such as data replication, distributed transactions, and conflict resolution.

These three properties are considered fundamental elements necessary when designing a system. They are also frequently mentioned in theorems and theories when building a standard system. For example, the CAP theorem; in which Partition Tolerance is the fault tolerance of the system, referring to the system's ability to ensure normal operation even if the connections between nodes in the system are broken.



Partition – Data partitioning

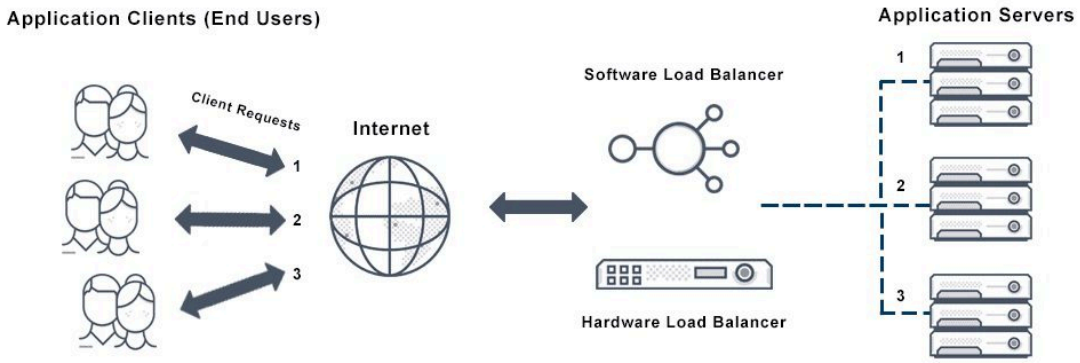
Partitioning is the process of dividing data into smaller, more manageable parts. Factors such as data access types, data distribution, and data replication are considered when you want to effectively partition data on a system.
In database systems, data partitioning is typically divided into two types:

- Horizontal sharding – also known as sharding – is a method of dividing rows of a table into smaller tables and storing them on servers.
- Vertically – this involves splitting the columns of a table into separate tables, reducing the number of columns to improve query performance.

Load Balancing

Load balancing is the process of distributing network traffic across multiple servers to prevent overload. Load balancing plays a particularly important role in situations where traffic spikes or when requests to servers are unevenly distributed.
To perform load balancing, systems engineers typically apply several algorithms to determine the distribution of incoming traffic, such as:

- **Round Robin**: sequential and even distribution of requests
- **Least Connections**: Assigns the request to the server with the least activity.
- **IP Hash**Use a hash function to identify the server for the request based on the client's IP address.



Caching – Memory Cache

Caching is a process of storing frequently accessed data in memory to facilitate faster access on subsequent occasions. A cache is a high-speed storage layer, typically located between the application and the data source. When the application requests data, the system first searches the cache; this high access speed saves time and improves processing efficiency. If the requested data is not found, the system then proceeds to search the data source.

When discussing System Design, it's impossible not to mention the popular architectural designs currently in use, including Monolith, SOA, and Microservices. Monolith is a traditional model suitable for small-scale projects. However, to meet the increasing demands and integration of various services, SOA and Microservices are now the preferred choices for modern systems.

Microservices

MicroservicesMicroservices is a way of organizing applications as a collection of loosely linked services that can be deployed independently. Each service in the system handles a specific function or domain within the application and can communicate with other services through predefined APIs. To build a microservices architecture, we

need to consider factors such as the boundaries between subservices and their communication capabilities.

SOA – Service-Oriented Architecture

Service-oriented architecture is a software design approach in which applications are built as a collection of services. The main difference between SOA and Microservices is their scope of application: while SOA is typically applied to enterprise-level applications where applications communicate with each other, Microservices are geared towards developing applications where communication occurs between components within an application.

DNS – Domain Name System